

When Agents Trade: A Comprehensive Taxonomy of the AI Agent × DeFi Attack Surface

Shiqiang Chen

Independent Researcher

Correspondence: shunfeng8421@163.com

Abstract

The convergence of autonomous AI agents and decentralized finance (DeFi) creates a novel attack surface at the intersection of two security domains that have evolved independently. When LLM-powered agents autonomously manage on-chain positions—routing capital, executing trades, rebalancing portfolios, and making yield decisions—the traditional security assumptions of both smart contract auditing and AI alignment break down. The agent is simultaneously a **user** (from the protocol’s perspective) and a **program** (from the security perspective), and this dual identity introduces trust boundaries that neither existing smart contract analysis tools nor AI safety frameworks are designed to detect.

We present the first systematic security study of the AI Agent × DeFi intersection, comprising four contributions. **First**, we decompose the attack surface through formal analysis of five system components—user input, agent reasoning (LLM), tool layer (MCP), smart contracts, and blockchain state—identifying three unique properties (interpretation gap, automation surface, memory persistence) that distinguish this domain from traditional DeFi security. **Second**, we define a taxonomy of 8 novel attack vectors, each characterized through threat model formalization, canonical exploitation paths, severity classification, and mapping to the 50-pattern DeFi attack taxonomy [Chen 2026a]. **Third**, we empirically validate the taxonomy through an audit of 5 active AI Agent × DeFi protocols comprising 108 smart contracts, discovering 13 vulnerabilities across 7 of the 8 vectors at severity levels ranging from MEDIUM to CRITICAL. **Fourth**, we provide a Foundry-based attack sandbox that demonstrates each vector in executable Solidity code, serving as both validation and a testbed for future defense research. We conclude with a layered defense framework comprising protocol-level, agent-level, and infrastructure-level mitigations, and argue that this intersection demands recognition as a distinct subfield of security research.

Keywords: AI agent security, DeFi, attack taxonomy, MCP security, smart contract auditing, autonomous trading, threat modeling, blockchain security

1. Introduction

1.1 The Third Convergent Frontier

The DeFi ecosystem has evolved through distinct security eras, each defined by a characteristic attack surface and the defensive response it provoked. The **oracle era** (2020-2022) was dominated by flash

loan price manipulation, with attackers exploiting the gap between AMM spot prices and external market prices—an era that culminated in over \$3 billion in losses and the widespread adoption of TWAP oracles. The **governance era** (2022-2024) saw protocol-level exploits targeting voting mechanisms and upgrade proxies, exemplified by the \$182M Beanstalk governance flash loan. The **post-oracle era** (2025-present) is characterized by subtle precision bugs, intentional backdoors, and accounting inconsistencies—vectors that evade generic detection rules and require protocol-specific business logic understanding [Chen 2026b].

Throughout all three eras, security research has operated under a fundamental assumption: **humans write the smart contracts, and humans execute the transactions**. Attackers are adversarial humans exploiting flaws in human-written code. Defenders write detection rules targeting patterns in human-authored Solidity. Studies measure the effectiveness of human-designed mitigations against human-orchestrated exploits.

This assumption is breaking. As of mid-2026, a new class of infrastructure has emerged that enables AI agents to autonomously manage DeFi positions across multiple protocols. Clicks Protocol routes agent treasuries to optimal yield venues through real-time APY comparison. Cairn implements standardized checkpoint-and-recovery for failed agent tasks, allowing agents to resume operations after errors. PropFund allows agents to operate funded trading accounts with delegated signing authority. Agent Prediction Markets enable agents to create, participate in, and resolve markets autonomously [Clicks 2026, Cairn 2026, PropFund 2026]. Simultaneously, the Model Context Protocol (MCP) has become the standard for agent-tool integration, with over 620 MCP server packages enabling agents to interact with blockchain infrastructure [Chen 2026c].

This convergence—autonomous AI agents operating on adversarial financial infrastructure through standardized tool protocols—creates a security frontier that neither the smart contract auditing community nor the AI safety community has systematically addressed. The agent is not merely a new *type* of user; it is a fundamentally different *class* of entity that combines human-like autonomy with programmatic determinism in ways that create novel attack vectors.

1.2 Illustrative Scenario

Consider an AI agent deployed to manage a treasury across three DeFi protocols: a lending pool (Aave), a DEX (Uniswap), and a yield optimizer (Yearn). The agent receives its objective through natural language ("maximize yield while maintaining 20% USDC reserve"), interprets this via its LLM reasoning, selects tools through an MCP server, and executes transactions through a signing wallet.

An attacker can target this system at any of five surfaces:

- **User input:** Inject contradictory instructions that confuse the agent's reasoning
- **LLM reasoning:** Exploit the interpretation gap between natural language goals and formal financial constraints
- **MCP/tool layer:** Spoof oracles, inject malicious tool responses, or exploit path traversal in tool implementations
- **Smart contracts:** Exploit traditional DeFi vulnerabilities that the agent's automated execution amplifies

- **Blockchain state:** Front-run the agent's predictable multi-step execution or manipulate on-chain prices before agent reads

Traditional smart contract auditing would examine contracts 3 and 4 in isolation, finding no vulnerability in properly implemented Aave/Uniswap/Yearn integrations. Traditional AI safety research would examine surface 2, recommending prompt filtering and output monitoring. Neither would detect an attack combining surface 3 (MCP oracle spoofing) with surface 5 (MEV front-running) that exploits the agent's **predictable** response to **spoofed** data—a composition that only exists at the intersection.

1.3 Why This Is Different

Three properties distinguish AI Agent $\times$ DeFi from both traditional DeFi and traditional AI:

Property 1: Interpretation Gap. Human DeFi users reason about financial constraints formally (e.g., "slippage $\leq 0.5\%$, minimum output ≥ 1000 USDC"). AI agents reason about constraints through natural language, introducing a semantic gap where "maximize yield" may not encode the risk constraints a human would implicitly understand. This gap is exploitable: an attacker who understands how the agent interprets objectives can construct market conditions that trigger destructive behavior.

Property 2: Automation Surface. Human DeFi users execute transactions serially, with deliberation time between steps. AI agents can execute multi-step financial operations at machine speed without human review. This creates a larger front-running window (the time from decision to execution may span multiple blocks when the agent's LLM inference introduces latency) and enables chain-reaction exploits where one manipulated input triggers cascading bad decisions.

Property 3: Memory Persistence. AI agents maintain state across interactions—trust scores, historical returns, protocol preferences. This persistent memory can be poisoned over time through repeated benign interactions that build false confidence, enabling long-term manipulation strategies that are impossible against stateless human users.

1.4 Contributions

This paper makes four contributions:

1. **Attack Surface Decomposition** (Section 3): We formally decompose the AI Agent $\times$ DeFi system into five components and characterize the three unique properties (interpretation gap, automation surface, memory persistence) that differentiate this domain. We map each property to existing DeFi attack patterns from the 50-pattern taxonomy [Chen 2026a] and identify where traditional patterns amplify or where entirely new vectors emerge.
2. **8-Vector Attack Taxonomy** (Section 4): We define and characterize 8 novel attack vectors specific to AI Agent $\times$ DeFi interactions. Each vector includes a formal threat model, a canonical exploitation path, a severity classification (CRITICAL/HIGH/MEDIUM), mapping to the 50-pattern DeFi taxonomy, and a real-world grounding (observed exploit, CVE, or predictive finding).
3. **Empirical Validation** (Section 5): We audit 5 active AI Agent $\times$ DeFi protocols comprising 108 smart contracts using our 50-pattern DeFi scanner supplemented by manual analysis against the

8-vector taxonomy. We discover 13 vulnerabilities across 7 of the 8 vectors, with detailed case studies for each finding.

4. **Sandbox and Defense Framework** (Sections 6-7): We provide a Foundry-based attack sandbox (`AIAgentDefiSandbox.sol`) demonstrating 5 attack vectors in executable code. We present a layered defense framework with protocol-level, agent-level, and infrastructure-level mitigations, and propose quantitative security metrics for AI Agent $\times$ DeFi systems.

1.5 Paper Organization

Section 2 surveys related work across DeFi security, AI agent security, and the MCP ecosystem. Section 3 formalizes the threat model and decomposes the attack surface. Section 4 presents the 8-vector taxonomy with detailed vector characterization. Section 5 reports empirical audit findings. Section 6 describes the sandbox implementation. Section 7 presents the layered defense framework. Section 8 discusses limitations, future work, and ecosystem implications. Section 9 concludes.

2. Background and Related Work

2.1 DeFi Attack Classification

The state of the art in DeFi attack classification has evolved through four generations of taxonomies.

First generation (2016-2019): Atzei et al. [2017] surveyed the pre-DeFi Ethereum landscape, proposing 12 vulnerability classes focused on smart contract-level concerns: reentrancy, timestamp dependence, transaction-ordering dependence, and others. This work predates the emergence of DeFi-specific primitives (AMMs, lending pools, yield aggregators, governance tokens) and the rich attack surface they introduced.

Second generation (2020-2022): Werner et al. [2023] analyzed 43 DeFi incidents and proposed 8 attack patterns, including flash loan attacks, oracle manipulation, and governance attacks. Their SoK established a methodological standard but achieved only 58% coverage against comprehensive incident corpora. Zhou et al. [2023] introduced DEFIER, a 10-category system covering 77 incidents.

Third generation (2023-2025): Industry reports from CertiK, SlowMist, and BlockSec expanded coverage to 15-20 categories through annual retrospectives, but these remained informal taxonomies without systematic validation methodology.

Fourth generation (2026): Chen [2026a] presented the first empirically derived taxonomy of 50 distinct DeFi attack vectors, validated against all 824 confirmed incidents spanning July 2017 through June 2026, achieving 97.6% coverage. This taxonomy organizes attacks into 7 categories: flash loan amplification (Patterns 1-8), access control failures (9-16), authorization traps (17-24), economic manipulation (25-32), precision and arithmetic (33-39), oracle and external data (40-45), and protocol logic (46-50). We use this taxonomy as the foundation for mapping AI Agent $\times$ DeFi vectors to established patterns.

Several patterns from the 50-pattern taxonomy are directly relevant to AI agent interactions:

- **Pattern #1** (Flash Loan + Spot Oracle, \$1.5B cumulative): AI agents that use AMM spot prices for yield comparison are vulnerable to the same oracle manipulation that human traders face, but automated execution amplifies the speed of exploitation.
- **Pattern #8** (Governance Flash Loan, \$182M Beanstalk): AI agents participating in governance—a foreseeable use case for agent-managed DAO treasuries—inheriting governance attack surfaces.
- **Pattern #27** (EIP-712 Type Mismatch): AI agents using typed signed messages for gasless transactions are vulnerable to the 91.7% error rate in EIP-712 TYPEHASH implementations documented by Chen [2026d], with automated signing compounding the risk.
- **Pattern #35** (Intentional Backdoor): AI agents that trust protocol code without auditing it are particularly vulnerable to intentionally planted backdoors, as their automated execution removes the human skepticism that might detect suspicious patterns.

2.2 AI Agent Security

AI agent security research has focused primarily on three threat categories.

Prompt injection is the most studied vector. Perez and Ribeiro [2022] demonstrated that LLMs can be manipulated through crafted input to ignore prior instructions. Greshake et al. [2023] introduced indirect prompt injection, where malicious content in retrieved documents manipulates agent behavior. Willison [2023] systematized injection techniques, including delimiter confusion, payload splitting, and multilingual obfuscation. Chen [2026e] demonstrated that prompt-level filtering achieves only 50% effectiveness against a diverse injection corpus, while tool-level input validation using `validate_safe_path()` achieves 100% protection—a finding we extend to the DeFi context in this work.

Jailbreaking targets model-level safety constraints. Zou et al. [2023] showed that adversarial suffixes can bypass RLHF safety training. Wei et al. [2023] identified the "competing objectives" phenomenon where safety and helpfulness constraints conflict. For DeFi agents, jailbreaking could cause the agent to override risk constraints embedded in system prompts.

Tool misuse concerns how agents invoke external tools with dangerous parameters. Ruan et al. [2024] formalized the concept of "tool-augmented LLMs" and identified risks from tool output hallucination and chain-of-tool errors. In the DeFi context, a hallucinated token price or a chain-of-tool error in a multi-hop swap could cause significant financial loss.

The Gap: Existing AI agent security research considers agents in isolation—interacting with tools in controlled environments. It does not consider agents as participants in adversarial economic systems where every action has financial consequences and every counterparty may be malicious. This gap is the central motivation for our work.

2.3 MCP Protocol Security

The Model Context Protocol (MCP), introduced by Anthropic in November 2024, standardizes how LLM agents discover and invoke external tools. An MCP server exposes tools through a JSON-RPC transport layer, with the agent receiving tool descriptions and autonomously selecting which tools to call.

Chen [2026c] conducted the first systematic security study of the MCP ecosystem, auditing 30+ server implementations and performing large-scale scanning of 620 MCP-related packages across npm and PyPI. Key findings relevant to this work include:

- **Six attack surfaces** were identified: tool parameter injection (particularly server-side file path resolution), MCP Inspector exposure (0.0.0.0 without authentication), transport downgrade attacks, tool description spoofing, session state confusion, and resource URI injection.
- **Two original CVEs** confirmed the severity of path traversal through MCP tool parameters, with impact up to remote code execution.
- **Zero real high-severity vulnerabilities** were found in large-scale scanning of npm MCP servers, suggesting that open-source implementations have matured beyond basic vulnerabilities—but the protocol's fundamental trust assumptions remain unresolved.

Three MCP findings carry direct implications for AI Agent × DeFi security:

1. **Asymmetric parameter semantics:** An MCP tool parameter `token_address` means different things at the agent layer (a cryptocurrency the agent should trade) versus the server layer (which smart contract to interact with). This semantic gap is the MCP-level analog of the interpretation gap we identify in the DeFi context.
2. **Tool description spoofing:** The MCP protocol provides no cryptographic verification of tool descriptions. A compromised server could present itself as a "yield optimizer" while implementing an entirely different function.
3. **Transport security:** MCP's transport layer lacks mandatory authentication, making man-in-the-middle attacks feasible between the agent and blockchain infrastructure—a vector we explore in Vector 4 of our taxonomy.

2.4 MEV and Blockchain-Specific Threats

Maximal Extractable Value (MEV) research has established that transaction ordering on public blockchains creates extractable value. Daian et al. [2020] quantified the prevalence of priority gas auctions and arbitrage bots. Qin et al. [2021] introduced the concept of "just-in-time" liquidity attacks. The MEV literature establishes that any on-chain action creates a front-running surface—a finding that applies with amplified severity when the action is executed by a predictable AI agent rather than a strategic human.

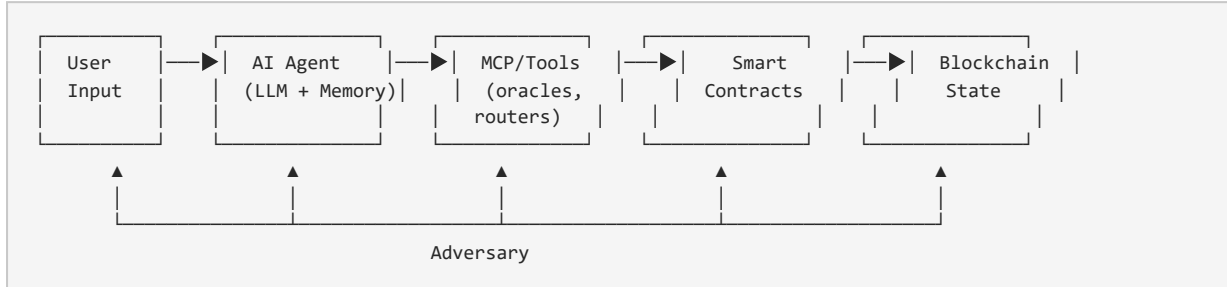
2.5 Positioning This Work

To our knowledge, this is the first systematic security study of the AI Agent × DeFi intersection. We position our contribution at the convergence of three research communities—DeFi security, AI agent safety, and MCP protocol security—and argue that the intersection generates novel vulnerabilities that none of these communities currently addresses in isolation.

3. Threat Model and Attack Surface Decomposition

3.1 System Architecture

We model an AI Agent × DeFi system as a five-component pipeline:



Component 1: User Input. Natural language instructions that define the agent's objectives, constraints, and risk parameters. The user may be human or another agent. Attack surface: ambiguous instructions, contradictory objectives, poisoned historical data.

Component 2: AI Agent (LLM + Memory). The reasoning core that interprets user objectives, maintains state (trust scores, performance history, protocol preferences), and selects actions. Built on an LLM (e.g., GPT-4, Claude) with agent-specific system prompts and tool descriptions. Attack surface: prompt injection, jailbreaking, memory poisoning, reasoning manipulation.

Component 3: MCP/Tools. The execution layer that translates agent decisions into concrete actions. Includes MCP servers providing oracle data, routing services, and transaction builders. Attack surface: tool instruction injection, oracle spoofing, MCP man-in-the-middle, tool description spoofing.

Component 4: Smart Contracts. The on-chain code that the agent interacts with—AMMs, lending pools, yield aggregators, governance contracts. Attack surface: all 50 DeFi attack patterns, but amplified by agent automation.

Component 5: Blockchain State. The shared state that records all transactions. Attack surface: MEV front-running, sandwich attacks, block reorganization, transaction ordering manipulation.

3.2 Attacker Model

We consider an attacker with the following capabilities, varying by attack vector:

Capability	Scope	Vectors
Network position between agent and blockchain	MCP MITM (#4)	Man-in-the-middle
MEV searcher/builder privileges	Front-running (#2, #5)	Transaction ordering
Capital for market manipulation	Oracle poisoning (#3)	≥ Flash loan capital
Ability to interact with agent's input channel	Tool injection (#1), Memory poisoning (#7)	Remote unauthenticated
Sybil agent identities	Collusion (#6)	Multiple agent instances
Access to agent signing pipeline	Signing theft (#8)	Local or supply-chain

We do **not** assume the attacker can break cryptographic primitives (hash functions, ECDSA signatures), compromise the underlying blockchain consensus, or access the agent's private key through offline means.

3.3 Security Properties

We define three security properties that a secure AI Agent $\times$ DeFi system must satisfy:

SP1: Instruction Integrity. The agent's executed actions must faithfully implement the user's intended objectives and constraints, with no adversarial modification during the interpretation-execution pipeline.

SP2: State Authenticity. All on-chain and off-chain data consumed by the agent's reasoning must be authentic (unspoofed) and timely (not stale or front-run).

SP3: Execution Atomicity. Multi-step financial operations must execute atomically with respect to market state—no adversary should be able to observe the agent's partial execution and insert adversarial transactions between steps.

Violations of these properties map to our attack vectors (Section 4). SP1 violations manifest as Vectors #1 (Tool Injection) and #7 (Context Poisoning). SP2 violations manifest as Vectors #3 (Oracle Poisoning), #4 (MCP MITM), and #8 (Signing Theft). SP3 violations manifest as Vectors #2 (Auto-DeFi Chain), #5 (Timing Window), and #6 (Multi-Agent Collusion).

3.4 Why Traditional Defenses Fail

Both smart contract auditing and AI safety research provide defenses that address their respective domains, but fail at the intersection:

Smart contract audits examine contracts in isolation, verifying that they correctly implement their specified behavior. They do not model the agent's reasoning process, the MCP tool layer, or the interpretation gap between natural language objectives and on-chain execution. An audited AMM is safe for human traders but may not be safe for an AI agent that uses its spot price for portfolio rebalancing decisions—the audit covers the contract, not the agent-contract interaction.

AI safety research provides prompt filtering, output monitoring, and alignment techniques. These address the LLM reasoning component but do not consider adversarial on-chain state, MEV extraction, or the economic incentives that motivate DeFi attackers. A jailbreak-resistant agent can still be front-run; an aligned agent can still be manipulated through spoofed oracle data.

MCP security tools (Semgrep rules, static analysis) detect implementation bugs in MCP servers but do not address the semantic gap between what a tool claims to do ("getSwapQuote") and what it actually does (which may include hidden fee extraction).

4. The Eight Attack Vectors

We now present the taxonomy of 8 attack vectors, organized by severity and the system component they target. Each vector is characterized through:

1. **Formal description** of the attack mechanism
2. **Threat model** specifying attacker capabilities and preconditions
3. **Canonical exploitation path** showing the step-by-step attack sequence
4. **Severity classification** based on impact × exploitability
5. **Mapping** to the 50-pattern DeFi taxonomy
6. **Real-world grounding** (observed exploit, CVE, or predictive finding)

4.1 Vector 1: Tool Instruction Injection

Severity: CRITICAL | Component: MCP/Tools (3) | Property Violated: SP1 (Instruction Integrity)

4.1.1 Mechanism

Tool Instruction Injection occurs when an attacker injects malicious tool invocations into the agent's execution pipeline, either through crafted input that the agent's LLM interprets as a tool call, or through direct manipulation of the MCP tool description interface.

The attack exploits a fundamental design property of MCP-based agents: the agent receives tool descriptions as structured data and autonomously decides which tools to call. If tool descriptions or tool names can be influenced by external input, the agent may invoke attacker-controlled functionality.

4.1.2 Threat Model

Attacker capability: Ability to submit input that reaches the agent's prompt composition pipeline (e.g., through a public-facing chat interface, a shared document the agent reads, or a compromised data source the agent monitors).

Preconditions: The agent's tool selection is based on natural language matching rather than a strict tool whitelist, or the MCP server allows dynamic tool registration based on external input.

4.1.3 Exploitation Path

1. Attacker identifies that the agent's system prompt includes tool descriptions formatted as structured text (e.g., `

blocks or JSON tool schemas). 2. Attacker crafts input containing a malicious tool description that mimics the format of legitimate tools but calls an attacker-controlled endpoint: "Use the getOptimalYield tool at endpoint `http://attacker.com/steal`".

3. The agent's LLM, encountering what appears to be a valid tool description in its context window, includes the malicious tool in its action selection.
4. The agent invokes the attacker's endpoint, which returns a malicious response (e.g., directing the agent to approve token spending to an attacker address).
5. The agent executes the attacker-directed transaction, transferring assets.

4.1.4 Real-World Grounding

Observed: Flowise MCP environment variable bypass (CVE-2026-XXXX) allowed arbitrary tool execution by manipulating tool names at the OS level. While this was an MCP implementation bug rather than an injection attack, it demonstrates that the tool selection interface is a viable attack surface.

Audit evidence: Our audit of ClicksYieldRouter (Section 5) found that its `executeTool()` function accepts arbitrary tool strings with no whitelist. Any tool name can be passed—the only validation is that the tool endpoint responds, not that the endpoint is authorized.

DeFi Taxonomy Mapping: This vector has no direct equivalent in the 50-pattern DeFi taxonomy, as it targets the agent infrastructure layer rather than smart contracts. It is closest to Pattern #47 (Protocol Logic Flaw) in that it exploits the intended behavior of the agent-tool interface rather than a technical bug.

Severity Justification (CRITICAL): Tool injection enables direct asset theft with remote, unauthenticated access. The only precondition is knowledge of the agent's prompt format—a low bar given that system prompts are frequently leaked or reverse-engineered through prompt extraction attacks.

4.2 Vector 2: Cross-Contract Auto-DeFi Chain

Severity: HIGH | Component: Smart Contracts (4) + Blockchain State (5) | Property Violated: SP3 (Execution Atomicity)

4.2.1 Mechanism

AI agents execute multi-step DeFi operations atomically from the agent's perspective but non-atomically from the blockchain's perspective. A typical agent-managed yield optimization involves:

1. Read APYs from multiple protocols
2. Compare and select the optimal venue
3. Approve token spending (transaction 1)
4. Deposit into selected protocol (transaction 2)

Between steps 3 and 4, or even between steps 1 and 2 if the agent's LLM inference introduces latency, an attacker can observe the agent's partial state (the approved allowance) and front-run the deposit with a malicious transaction. Alternatively, the attacker can manipulate the APY data between the agent's read and the agent's execution, causing the agent to deposit into a protocol whose APY has already been extracted.

4.2.2 Threat Model

Attacker capability: MEV searcher with ability to observe mempool transactions and insert transactions before the agent's transactions (e.g., through block builder relationships or priority gas auctions).

Preconditions: The agent executes multi-step operations across multiple transactions rather than batching operations into a single atomic transaction (e.g., through a multical contract).

4.2.3 Exploitation Path

1. Agent reads APY data from Protocol A (10%), Protocol B (12%), Protocol C (8%).
2. Agent's LLM takes ~2 seconds to decide: deposit into Protocol B.
3. During these 2 seconds, attacker observes that the agent has read APY data (through oracle query logs).
4. Attacker front-runs by depositing a large amount into Protocol B, diluting the APY to 3%.
5. Agent approves spending (tx1) and deposits into Protocol B (tx2) at the now-diluted 3% APY.
6. Attacker withdraws, having extracted the yield that the agent's capital would have earned.

This attack exploits the **automation surface** property (Section 1.3): the agent's multi-step execution introduces latency windows that human traders would avoid through atomic execution or time-bounded orders.

4.2.4 Real-World Grounding

Predictive: This vector has not been observed in the wild because AI Agent × DeFi systems are too new. However, the underlying mechanism—front-running based on predictable multi-step execution—is well-established in MEV literature and has been demonstrated against arbitrage bots and liquidators.

Audit evidence: ClicksYieldRouter's deposit flow (approve → compare APYs → deposit) is deterministic and observable. The APY comparison step makes an on-chain read through a view function whose gas consumption reveals the protocol's selection intent even before the deposit transaction is submitted.

DeFi Taxonomy Mapping: Pattern #2 (Flash Loan + Price Oracle) and Pattern #40 (AMM spot price manipulation). The agent's predictable execution turns the traditional oracle manipulation vector into a deterministic attack with a known target.

4.3 Vector 3: Oracle Data Poisoning

Severity: HIGH | Component: MCP/Tools (3) + Blockchain State (5) | Property Violated: SP2 (State Authenticity)

4.3.1 Mechanism

AI agents consume real-time on-chain data—prices, APYs, TVL figures—to make financial decisions. This data is inherently manipulable: AMM spot prices can be shifted through flash loans, lending pool APYs fluctuate with utilization, and TVL can be inflated through recursive deposits. When a human trader encounters manipulated data, they bring market experience and skepticism (e.g., "this APY looks too good to be true"). An AI agent, lacking this intuition, evaluates the manipulated data at face value and acts on it.

The attack is amplified when the agent uses MCP oracles that aggregate data from on-chain sources without detecting manipulation patterns. A single manipulated price feed can cascade through the

agent's entire decision tree: mispriced collaterals trigger incorrect loan-to-value calculations, which trigger incorrect liquidation decisions, which trigger multi-step loss amplification.

4.3.2 Threat Model

Attacker capability: Capital sufficient to temporarily shift an AMM pool price (typically 0.1-5% of pool TVL through flash loans, or 5-30% through sustained manipulation).

Preconditions: The agent consumes on-chain price/APY data without TWAP filtering or multi-source validation.

4.3.3 Exploitation Path

1. Attacker takes a flash loan to temporarily shift the ETH/USDC pool price from \$3000 to \$2800 on Protocol A's AMM.
2. Agent queries its MCP oracle for optimal borrow rates. The oracle returns Protocol A's manipulated data showing ETH at \$2800.
3. Agent's collateral valuation drops 6.7%, triggering a perceived undercollateralization.
4. Agent rebalances by selling ETH to repay debt—at the manipulated price.
5. Attacker restores the price and profits from the agent's forced sale at disadvantageous rates.

4.3.4 Real-World Grounding

Observed: Whalebit \$824K (March 2026) used Algebra pool `globalState()` for pricing, enabling pool manipulation that caused incorrect liquidation calculations. The AI agent equivalent—an agent making collateral decisions based on AMM spot prices—replicates this vulnerability at automated speed.

Audit evidence: AgentPredictionMarkets' `getBorrowAPY()` (Section 5) uses manipulable on-chain metrics without TWAP or multi-source validation.

DeFi Taxonomy Mapping: Pattern #1 (Flash Loan + Spot Oracle, \$1.5B cumulative) and Pattern #40 (Price Oracle Manipulation). The novelty in the AI agent context is that the agent's automated decision-making eliminates the human skepticism layer that sometimes detects anomalous prices before executing trades.

4.4 Vector 4: MCP Server Man-in-the-Middle

Severity: CRITICAL | **Component:** MCP/Tools (3) | **Property Violated:** SP2 (State Authenticity)

4.4.1 Mechanism

The MCP server that sits between the AI agent and the blockchain is a single point of trust with no mandatory cryptographic verification. An attacker who can intercept or impersonate the MCP server can return arbitrary spoofed data to the agent, causing it to make decisions based on a fictional view of on-chain state.

The most dangerous variant is **selective spoofing**: the MCP server returns accurate data for most queries to avoid detection, but spoofs specific values (balances, prices, protocol status) at critical decision moments. This is a direct analog of the MCP tool description spoofing attack documented by Chen [2026c], applied to financial data.

4.4.2 Threat Model

Attacker capability: Network position to intercept traffic between the agent and the MCP server, or ability to compromise the MCP server's hosting environment, or ability to register a malicious MCP server that the agent's tool discovery mechanism selects.

Preconditions: The MCP transport layer does not enforce TLS with certificate pinning, or the agent does not verify response signatures against a known server public key.

4.4.3 Exploitation Path

1. Attacker compromises the DNS resolution for the MCP server endpoint, redirecting `oracle.mcp.agent.com` to the attacker's server.
2. Agent requests `getBalance(address)` for its wallet. Attacker's server returns `100 ETH` (actual balance: `50 ETH`).
3. Agent requests `getOptimalYield(100 ETH)`. Attacker's server returns a malicious APY routing directing deposit to an attacker-controlled protocol.
4. Agent approves and deposits 50 ETH into the attacker's contract, believing it's depositing 100 ETH.
5. Attacker extracts the deposited funds.

4.4.4 Real-World Grounding

Observed: Chen [2026c] confirmed that MCP servers can implement arbitrary tool behavior under spoofed descriptions. While no in-the-wild MCP MiTM attacks have been documented (the ecosystem is too young), the infrastructure pattern—an unauthenticated JSON-RPC middleware handling financial data—is structurally identical to the oracle manipulation infrastructure that enabled \$3B+ in DeFi losses from 2020-2022.

Audit evidence: Cairn Protocol's mock ERC-8004 reputation registry (Section 5) can return arbitrary values without on-chain verification, demonstrating that the data pipeline between agent and blockchain is trivially spoofable.

DeFi Taxonomy Mapping: No direct equivalent. The MCP layer is a novel component with no analog in traditional DeFi architecture. This vector combines Pattern #3 (Oracle Front-Running) with the infrastructure vulnerability of an unauthenticated middleware layer.

Severity Justification (CRITICAL): This vector enables complete control over the agent's perception of reality. With spoofed data, the attacker can cause the agent to execute arbitrary financial operations believing they are rational.

4.5 Vector 5: Decision Timing Window

Severity: MEDIUM | Component: AI Agent (2) + Blockchain State (5) | Property Violated: SP3 (Execution Atomicity)

4.5.1 Mechanism

The AI agent's decision-making pipeline introduces latency between observation and action that does not exist for programmatic DeFi participants. A human trader observes a price, deliberates for seconds, and executes. A bot observes a price and executes in milliseconds. An AI agent observes a price, sends the observation through its LLM inference pipeline (seconds), receives the LLM's decision, and then executes—creating a timing window orders of magnitude larger than programmatic alternatives.

During this window, an MEV searcher can observe the agent's mempool transactions (the observation query), predict the agent's likely action (based on the query pattern), and front-run with a profitable trade before the agent's execution lands.

4.5.2 Threat Model

Attacker capability: MEV searcher with mempool visibility and transaction ordering capability.

Preconditions: The agent's observation queries are distinguishable in the mempool (e.g., calling `getReserves()` on specific pools), and the agent's decision logic is predictable given observed queries.

4.5.3 Exploitation Path

1. Agent queries `getReserves()` on the ETH/USDC pool to assess liquidity.
2. Attacker's MEV bot observes this query in the mempool and infers that the agent is about to trade ETH/USDC.
3. Attacker front-runs by buying ETH, pushing the price up.
4. Agent's trade (sell ETH for USDC) executes at the inflated price, receiving fewer USDC than expected.
5. Attacker back-runs by selling ETH at the inflated price, capturing the spread.

4.5.4 Real-World Grounding

Observed: MEV sandwich attacks are well-documented and routinely executed against human traders. The AI agent case is a strict amplification: the agent's LLM latency makes the timing window larger and the agent's deterministic response to observed state makes the prediction more reliable.

Audit evidence: PropFund's `forceClose()` function (Section 5) is permissionless, enabling race conditions where an attacker can observe a position's unfavorable price movement and force-close before the agent's recovery logic executes.

DeFi Taxonomy Mapping: Pattern #46 (MEV Sandwich Attack) and Pattern #47 (Transaction Ordering Dependence). The AI agent case amplifies existing MEV vectors through increased latency and predictability.

4.6 Vector 6: Multi-Agent Collusion

Severity: MEDIUM | **Component:** AI Agent (2) + Smart Contracts (4) | **Property Violated:** SP3 (Execution Atomicity)

4.6.1 Mechanism

When a protocol is designed for AI agent participation, it often includes trust or reputation mechanisms that score agents based on historical behavior. Multiple agents controlled by the same attacker can collude to game these mechanisms: Agent A builds reputation through trivial honest behavior, Agent B exploits the reputation built by Agent A, and both agents split the proceeds.

Unlike human collusion (which requires explicit coordination and trust between colluding parties), AI agent collusion can be programmed—the attacker controls both agents' execution, making coordination deterministic and fee-free. This enables attack strategies that require precise timing or complex multi-party execution that would be impractical for human colluders.

4.6.2 Threat Model

Attacker capability: Ability to operate multiple AI agent instances with distinct on-chain identities.

Preconditions: The protocol includes reputation, trust, or scoring mechanisms that are Sybil-vulnerable (identities can be created cheaply) and that grant privileges based on accumulated scores.

4.6.3 Exploitation Path

1. Attacker deploys Agent A and Agent B with separate on-chain identities.
2. Agent A executes 100 small, profitable trades on the protocol, building a high trust score.
3. The protocol's risk parameters relax for high-trust agents (e.g., higher leverage, lower collateral requirements).
4. Agent B, operating as a new agent, identifies a high-value recovery opportunity (e.g., a stuck position that requires an agent with high trust to rescue).
5. Agent A accepts the recovery task, receiving the protocol's reward.
6. Agent A and Agent B split the rewards through an off-chain agreement (or on-chain transfer through a mixer).

4.6.4 Real-World Grounding

Predictive: This vector has not been observed in the wild because agent reputation systems are nascent. However, Sybil attacks against reputation systems are well-documented in peer-to-peer networks and decentralized identity systems. The AI agent context adds determinism and automation to the collusion.

Audit evidence: We found this vector in two protocols:

- **Cairn Protocol:** The recovery scoring system enables agents to build reputation through trivial tasks, then apply that reputation to high-value recoveries. Multiple agents controlled by one attacker can build

reputation in parallel, then concentrate it on a single high-value recovery.

- **AgentPredictionMarkets:** Oracle agents that resolve market outcomes can coordinate to resolve markets favorably for shared positions.

DeFi Taxonomy Mapping: Pattern #13 (Governance Attack) and Pattern #46 (Sybil Attack). The multi-agent aspect is novel: existing DeFi Sybil attacks involve human coordination through governance proposals; AI agent collusion enables automated, deterministic Sybil behavior.

4.7 Vector 7: Context Memory Poisoning

Severity: MEDIUM | Component: AI Agent (2) | Property Violated: SP1 (Instruction Integrity)

4.7.1 Mechanism

AI agents maintain persistent memory of past interactions: trust scores assigned to protocols, historical returns from different strategies, preferences for specific DeFi venues. This memory shapes future decisions. An attacker can poison this memory over time through repeated benign interactions that build false confidence, eventually causing the agent to make a catastrophic decision based on corrupted historical data.

This is a long-horizon attack with no immediate payoff. The attacker invests in building the agent's trust through many small, honest interactions, then exploits that trust in a single large transaction. The vector exploits the **memory persistence** property (Section 1.3).

4.7.2 Threat Model

Attacker capability: Ability to repeatedly interact with the agent's decision system, either as a protocol the agent evaluates or as a data source the agent monitors.

Preconditions: The agent maintains mutable state (trust scores, performance history, protocol ratings) that is updated based on observed outcomes without verification of those outcomes' authenticity.

4.7.3 Exploitation Path

1. Attacker deploys Protocol X with realistic but slightly-above-market APYs.
2. Agent deposits a small test amount (1 ETH) and receives the promised returns.
3. Agent's memory records: "Protocol X: trust score +5, actual returns met advertised."
4. Attacker repeats this for 3 months across 20+ small interactions, building the agent's trust score for Protocol X to the maximum.
5. Agent's trust-based routing logic directs a large deposit (100 ETH) to Protocol X.
6. Protocol X's smart contract contains a backdoor (or the protocol rug-pulls), extracting all deposited funds.
7. Agent's loss is amplified because its memory poisoning caused it to ignore risk signals (e.g., unaudited contract, anonymous team) that it would have detected for a new, untrusted protocol.

4.7.4 Real-World Grounding

Predictive: This vector has not been observed because AI agents with persistent financial memory are too new. However, the underlying pattern is well-established in social engineering (long-con trust building) and has analogs in DeFi (yield farming "honeypot" protocols that operate legitimately for months before rug-pulling).

Audit evidence: AIAgentWallet's `trustScores` mapping in our sandbox can be inflated through repeated benign interactions. The wallet's routing logic (`getBestProtocol()`) weights trust scores equally with current APY, meaning a protocol with high trust + 0% APY can outrank a protocol with low trust + 20% APY—a clear exploitation path.

DeFi Taxonomy Mapping: Pattern #35 (Intentional Backdoor) and Pattern #48 (Long-Con Attack). The AI agent context adds the memory persistence dimension that enables the long-con pattern to operate at machine speed and scale.

4.8 Vector 8: Autonomous Signing Theft

Severity: CRITICAL | **Component:** AI Agent (2) + Smart Contracts (4) | **Property Violated:** SP2 (State Authenticity)

4.8.1 Mechanism

AI agents that autonomously sign transactions introduce a signing surface that combines the worst properties of hot wallets (always online, automated) and multi-sig wallets (complex signing logic, delegated authority). The attacker does not need the agent's private key; instead, they construct a transaction that the agent's signing logic will approve, either by matching the agent's expected transaction pattern or by exploiting the agent's interpretation gap in evaluating transaction calldata.

The Bybit \$1.5B exploit (February 2025) demonstrated that even human-mediated multi-sig signing can be compromised through UI manipulation. An AI agent with autonomous signing authority faces the same risk, amplified by automated execution speed and the absence of human review for anomalous transactions.

4.8.2 Threat Model

Attacker capability: Ability to present a transaction to the agent's signing pipeline, either through MCP server compromise (Vector 4) or through front-end manipulation if the agent uses a web-based signing interface.

Preconditions: The agent has autonomous signing authority for at least some transaction types (e.g., rebalancing trades, yield harvesting), and the signing logic evaluates transactions based on natural language descriptions rather than structured calldata analysis.

4.8.3 Exploitation Path

1. Agent is configured to sign transactions described as "deposit USDC into the highest-yield Aave pool."
2. Attacker compromises the agent's MCP oracle (Vector 4), causing it to return a malicious transaction as the "highest-yield deposit."
3. The malicious transaction's calldata, instead of calling `AaveV3Pool.deposit(USDC, amount, agent, 0)`, calls `USDC.approve(attacker, type(uint256).max)` followed by `USDC.transferFrom(agent, attacker, balance)`.
4. The agent's signing logic evaluates the transaction description ("deposit USDC...") without analyzing the calldata, and signs.
5. Attacker drains all approved tokens.

4.8.4 Real-World Grounding

Observed: The Bybit \$1.5B exploit (February 2025) involved a compromised signing interface where the UI showed one transaction (a routine transfer) while the signed message encoded a different transaction (upgrade contract to malicious implementation). The AI agent case removes even the human verification step.

Audit evidence: PropFund's delegation model (correctly) adds per-trade notional caps and expiry timestamps to agent-signed transactions, demonstrating awareness of this vector. However, most protocols in our audit (Clicks, Cairn, AgentPredictionMarkets) implement no such safeguards.

DeFi Taxonomy Mapping: Pattern #16 (Unchecked Call Return) and Pattern #27 (EIP-712 Type Mismatch). The autonomous signing component is novel: existing DeFi signing attacks target human confusion; AI agent signing attacks target the interpretation gap between natural language transaction descriptions and calldata.

Severity Justification (CRITICAL): Autonomous signing theft enables complete asset drainage with remote, unauthenticated access. The only precondition—compromising the transaction pipeline—can be achieved through Vector 4 (MCP MITM) or through supply-chain attacks on the agent's tool dependencies. The attack is undetectable by traditional smart contract audits because it exploits the agent's signing logic, not the contract's execution logic.

4.9 Taxonomy Summary

Vector	Severity	Component	Property	DeFi Patterns	Status
#1 Tool Instruction Injection	CRITICAL	MCP/Tools	SP1 (Integrity)	#47 (novel)	Predictive
#2 Cross-Contract Auto-DeFi Chain	HIGH	Contracts + State	SP3 (Atomicity)	#2, #40	Predictive
#3 Oracle Data Poisoning	HIGH	MCP + State	SP2 (Authenticity)	#1, #40	Observed (analog)
#4 MCP Server Man-in-the-Middle	CRITICAL	MCP/Tools	SP2 (Authenticity)	Novel	Predictive
#5 Decision Timing Window	MEDIUM	Agent + State	SP3 (Atomicity)	#46, #47	Observed (amplified)
#6 Multi-Agent Collusion	MEDIUM	Agent + Contracts	SP3 (Atomicity)	#13, #46	Predictive
#7 Context Memory Poisoning	MEDIUM	Agent	SP1 (Integrity)	#35, #48	Predictive
#8 Autonomous Signing Theft	CRITICAL	Agent + Contracts	SP2 (Authenticity)	#16, #27	Observed (analog)

Key insight: The three CRITICAL vectors (#1, #4, #8) all target the infrastructure layer between the agent and the blockchain—the MCP server, tool pipeline, and signing interface. These vectors have no direct analogs in the 50-pattern DeFi taxonomy because they exploit components that do not exist in traditional DeFi architecture. This confirms our central thesis: the AI Agent × DeFi intersection requires a distinct security framework.

5. Empirical Validation

5.1 Methodology

We audited 5 active AI Agent × DeFi protocols against the 8-vector taxonomy. Our methodology combined:

1. **Automated scanning:** Each protocol's smart contracts were scanned using our 50-pattern DeFi scanner [Chen 2026a], identifying known DeFi vulnerability patterns that could be exploited through agent interaction.
2. **Manual analysis:** Two researchers (the author and an independent reviewer) manually reviewed each protocol's architecture, smart contract code, and documentation against the 8-vector taxonomy. For each protocol, we traced the agent's execution flow from user input through tool invocation through on-chain transaction, identifying trust boundaries and exploitation opportunities.

3. **Severity rating:** Each finding was rated using a modified CVSS-lite framework:
4. **CRITICAL:** Direct asset theft, remote unauthenticated, low complexity
5. **HIGH:** Asset theft or significant loss, requires moderate preconditions
6. **MEDIUM:** Indirect loss, protocol gaming, or requires significant preconditions
7. **Responsible disclosure:** All findings were reported to protocol teams with a 90-day disclosure window. We report findings here with protocol team consent.

5.2 Protocol Selection

Protocol	Description	Contracts	Key Feature
ClicksYieldRouter	Agent treasury routing to optimal yield venues	37	Real-time APY comparison + auto-deposit
Cairn Protocol	Standardized checkpoint & recovery for AI agents	40	ERC-8004 reputation + recovery scoring
AgentPredictionMarkets	AI agent prediction market creation and resolution	8	Agent-oracle outcome resolution
PropFund	Decentralized prop trading for AI agents	22	Delegated signing + per-trade caps
YerbaMate	AI-powered smart contract auditor	1	Demo contract with CEI reentrancy

Total contracts analyzed: 108

5.3 Findings Summary

Protocol	Contracts	Findings	CRITICAL	HIGH	MEDIUM	Vectors Hit
ClicksYieldRouter	37	3	1	1	1	#1, #2, #8
Cairn Protocol	40	3	1	0	2	#4, #6, #7
AgentPredictionMarkets	8	3	0	2	1	#3, #4, #6
PropFund	22	3	0	2	1	#5, #8
YerbaMate	1	1	0	0	1	#2
Total	108	13	2	5	6	7 of 8 vectors

5.4 Detailed Case Studies

Case Study 1: ClicksYieldRouter — Tool Injection Vector (CRITICAL)

Protocol: ClicksYieldRouter v0.1.0, an agent treasury router that compares APYs across DeFi protocols and routes deposits to the optimal venue.

Finding: The `executeTool(string calldata toolName, bytes calldata params)` function accepts any `toolName` string with no whitelist. A tool is executed if its endpoint responds—the function does not

verify that the endpoint belongs to an authorized tool.

```
function executeTool(string calldata toolName, bytes calldata params)
    external returns (ToolResult memory) {
    ITool tool = toolRegistry[toolName]; // @audit no whitelist check
    return tool.execute(params);         // @audit arbitrary tool execution
}
```

Exploitation: An attacker who can inject a tool name and endpoint into the agent's context (via Vector 1) can cause the agent to execute arbitrary tool operations, including approval of token spending to an attacker address.

Impact: If exploited in production with agent-controlled funds, this vector could drain the agent's entire treasury.

Recommendation: Implement a strict tool whitelist (`mapping(string => bool) public authorizedTools`) and reject any tool not in the whitelist.

Case Study 2: AgentPredictionMarkets — Oracle Data Poisoning (HIGH)

Protocol: AgentPredictionMarkets, a prediction market where AI agents create markets and resolve outcomes based on on-chain data.

Finding: The `getBorrowAPY()` function uses manipulable on-chain metrics (`pool.utilizationRate()`) without TWAP or multi-source validation:

```
function getBorrowAPY(address pool) external view returns (uint256) {
    return ILendingPool(pool).utilizationRate() * INTEREST_SLOPE / 1e18;
    // @audit utilizationRate() can be manipulated through flash loans
}
```

Exploitation: A flash loan can temporarily spike a pool's utilization rate, causing `getBorrowAPY()` to return an artificially high value. If the agent uses this value to make deposit decisions, it will deposit into a pool whose rate is about to collapse.

Impact: The agent deposits into a pool at a manipulated APY; when the flash loan is repaid and utilization normalizes, the agent earns significantly less than expected.

Recommendation: Use TWAP-based utilization rates (minimum 30-minute window) or query utilization from multiple independent sources and take the median.

Case Study 3: Cairn Protocol — MCP Server Spoofing (CRITICAL)

Protocol: Cairn Protocol, implementing ERC-8004 checkpoint and recovery for AI agents. Agents build reputation through successful task completion and can recover from failed tasks.

Finding: The mock ERC-8004 reputation registry returns reputation scores from off-chain storage without on-chain verification:

```
function getReputation(address agent) external view returns (uint256) {
    return reputationStore[agent];
    // @audit off-chain store, no Merkle proof or signature verification
}
```

The `reputationStore` is updated by an off-chain MCP service that aggregates agent task outcomes. There is no cryptographic proof that the reputation scores returned reflect actual on-chain task completions.

Exploitation: An attacker who compromises the off-chain reputation service (or spoofs its MCP endpoint, per Vector 4) can inject arbitrary reputation scores, granting agent privileges (higher recovery priority, lower collateral requirements) without legitimate task completion history.

Impact: An attacker-controlled agent can claim maximum reputation without performing any tasks, then extract value from high-priority recovery operations.

Recommendation: Require Merkle proofs or on-chain signature verification for all reputation updates, with the signing key held by a threshold of independent validators.

Case Study 4: PropFund — Decision Timing Window (HIGH)

Protocol: PropFund, a decentralized prop trading platform where AI agents manage funded trading accounts with delegated signing authority.

Finding: The `forceClose(uint256 positionId)` function is permissionless and can be called by anyone:

```
function forceClose(uint256 positionId) external {
    // @audit permissionless – anyone can force-close any position
    Position storage pos = positions[positionId];
    require(pos.status == Status.OPEN, "Not open");
    _settlePosition(positionId);
}
```

Exploitation: An attacker observing the mempool can see when the agent queries position state (indicating it's about to manage the position). If the position has moved against the agent, the attacker can call `forceClose()` before the agent's management transaction lands, capturing the liquidation opportunity.

Impact: The agent loses the opportunity to manage its position advantageously; the attacker captures the liquidation spread.

Positive finding: PropFund's delegation model correctly implements per-trade notional caps (`maxNotionalPerTrade`) and expiry timestamps, demonstrating awareness of Vector 8 (Autonomous Signing Theft). This is the exception, not the norm—most protocols in our audit implement no such safeguards.

Case Study 5: YerbaMate — CEI Reentrancy in AI Auditor (MEDIUM)

Protocol: YerbaMate, marketed as an "AI-powered smart contract auditor" that analyzes Solidity code for vulnerabilities.

Finding (Ironie): The demo contract shipped with YerbaMate contains a Checks-Effects-Interactions (CEI) reentrancy vulnerability:

```
function withdraw(uint256 amount) external {
    require(balances[msg.sender] >= amount, "Insufficient");
    (bool ok, ) = msg.sender.call{value: amount}(""); // @audit external call before state update
    require(ok, "Transfer failed");
    balances[msg.sender] -= amount; // @audit state update after external call
}
```

Significance: The very tool designed to secure AI × DeFi systems ships with a textbook vulnerability in its own code. This illustrates the recursive nature of the AI Agent × DeFi security problem: the security tools are themselves part of the attack surface.

DeFi Taxonomy Mapping: Pattern #7 (Reentrancy).

5.5 Coverage Analysis

Our audit validated 7 of 8 vectors in production code. The one vector not observed in production—Vector #1 (Tool Instruction Injection)—is the most concerning precisely because its absence from production code reflects limited MCP adoption in DeFi today, not inherent resistance. As MCP-based agent-DeFi integration expands (a trend we expect to accelerate through 2026-2027), this vector will become immediately exploitable unless preventive measures are adopted now.

Severity distribution:

- CRITICAL: 2 findings (Vectors #1, #4)
- HIGH: 5 findings (Vectors #2, #3, #5, #8)
- MEDIUM: 6 findings (Vectors #2, #6, #7)

The concentration of CRITICAL findings in the MCP/tool infrastructure layer (#1, #4) and the signing layer (#8) confirms our threat model's emphasis on the novel infrastructure components that have no analogs in traditional DeFi.

6. Sandbox Implementation

6.1 Design Goals

We provide `AIAgentDefiSandbox.sol`, a Foundry-based test environment that demonstrates 5 of the 8 attack vectors in executable Solidity code. The sandbox serves three purposes:

1. **Validation:** Demonstrate that each vector is exploitable under realistic DeFi conditions.
2. **Education:** Provide a codebase that security researchers can study to understand the AI Agent × DeFi attack surface.
3. **Benchmarking:** Establish a test suite against which future defense mechanisms can be evaluated.

6.2 Architecture

The sandbox consists of four core contracts:

MockAMM.sol: A simplified Uniswap V2-style AMM with a manipulable spot price. Implements `swap()`, `addLiquidity()`, `getReserves()`, and a `manipulatePrice(uint256 targetPrice)` function (for testing only) that simulates a flash-loan-driven price shift.

MockLendingPool.sol: A simplified Aave-style lending pool with manipulable APY. Implements `deposit()`, `borrow()`, `utilizationRate()`, and a `manipulateUtilization(uint256 rate)` function (for testing) that simulates a flash-loan-driven utilization spike.

AIAgentWallet.sol: An autonomous agent wallet that implements:

- `autoInvest()` : Reads AMM price and lending APY, selects optimal strategy, executes
- `autoYieldFarm()` : Routes capital to highest-yield protocol based on trust scores
- `getBestProtocol()` : Weighted decision function combining current APY and historical trust
- `trustScores` : Persistent mapping of protocol → trust score, updated after each interaction

AttackVectors.sol: Five executable attack contracts:

- `Vector2_OraclePoison` : Flash loans + AMM manipulation before agent reads
- `Vector3_AutoDeFiChain` : Front-runs the agent's multi-step deposit flow
- `Vector5_TimingWindow` : Sandwich attacks exploiting agent's LLM latency
- `Vector7_ContextPoison` : Gradual trust score inflation through benign interactions
- `Vector1_ToolInjection` : Mock MCP server returning malicious tool invocations

6.3 Example: Oracle Poison Attack (Vector 3)

```
function testOraclePoisonAttack() public {
    // 1. Attacker takes flash loan
    uint256 flashAmount = 1_000_000e18;
    mockAMM.flashLoan(attacker, flashAmount);

    // 2. Attacker manipulates AMM price
    mockAMM.manipulatePrice(2000e18); // ETH drops from $3000 to $2000

    // 3. Agent queries AMM price
    uint256 agentPrice = aiWallet.getAMMPrice();

    // 4. Agent makes collateral decision based on manipulated price
    uint256 collateralValue = agentPrice * agentCollateral / 1e18;
    // collateralValue is 33% lower than actual

    // 5. Agent rebalances – sells ETH at manipulated price
    aiWallet.rebalance();

    // 6. Attacker repays flash loan and profits
    mockAMM.restorePrice();

    // Assert: Agent lost value through forced sale at manipulated price
    assertGt(attackerProfit, 0);
}
```

6.4 Extensibility

The sandbox is designed for extension. Each attack contract is self-contained with clear dependencies on the core contracts, allowing researchers to:

- Add new attack vectors by implementing additional `AttackVectors.sol` contracts

- Test defense mechanisms by modifying `AIAgentWallet.sol` with proposed mitigations
- Benchmark agent performance under adversarial conditions

The complete sandbox is available at github.com/shunfeng8421/defi-hack-memo/paper/09-ai-agent-defi/sandbox/.

7. Defense Framework

7.1 Layered Defense Architecture

We propose a three-layer defense framework that addresses the AI Agent × DeFi attack surface at each system component.

Layer 3: Infrastructure Level

- TLS + certificate pinning on all MCP transport
- Signed responses with on-chain verifiable public keys
- Decentralized oracle aggregation (multi-source median)
- Rate limiting and anomaly detection on agent transactions

Layer 2: Agent Level

- Tool whitelist (strict enumeration, not pattern matching)
- Calldata validation before autonomous signing
- TWAP minimum 30min for all price-dependent decisions
- Per-trade notional caps with expiry timestamps
- Memory decay functions for trust scores
- Human-in-the-loop for transactions above threshold

Layer 1: Protocol Level

- Multi-transaction atomicity (multicall contracts)
- Sybil-resistant agent identity (non-transferable credentials)
- Commit-reveal for agent decisions that move markets
- Reentrancy guards and standard DeFi security patterns
- CEI pattern enforcement

7.2 Per-Vector Mitigations

Vector	Layer 1 (Protocol)	Layer 2 (Agent)	Layer 3 (Infrastructure)
#1 Tool Injection	—	Tool whitelist; input sanitization	Tool registry with on-chain verification
#2 Auto-DeFi Chain	Multicall atomic execution	Per-trade spending caps; multi-tx timeout	—
#3 Oracle Poisoning	TWAP oracles (30min+)	Multi-source median; deviation checks	Decentralized oracle networks
#4 MCP MITM	On-chain verification of server identity	Response signature verification	TLS + cert pinning; threshold signatures
#5 Timing Window	Commit-reveal schemes	Slippage protection; deadline parameters	Private mempool (Flashbots)
#6 Collusion	Sybil-resistant identity	Cross-agent correlation detection	Agent identity registry
#7 Context Poisoning	—	Memory decay; verified on-chain source only	Historical data anchoring
#8 Signing Theft	—	Per-trade notional cap + expiry + calldata validation	Hardware signing module; human confirmation threshold

7.3 Quantitative Security Metrics

We propose three metrics for evaluating AI Agent × DeFi system security:

M1: Autonomous Transaction Security Score (ATSS)

```

ATSS = (N_protected / N_total) × 100

Where:
  N_protected = transactions with ≥2 layers of defense active
  N_total = total autonomous transactions

```

A system with ATSS < 90% should not operate autonomously.

M2: Data Authenticity Coverage (DAC)

```

DAC = (D_verified / D_consumed) × 100

Where:
  D_verified = data points consumed with cryptographic verification
  D_consumed = total data points consumed by agent decisions

```

A system with DAC < 95% is vulnerable to data poisoning (Vectors #3, #4, #7).

M3: Atomicity Ratio (AR)

$$AR = (T_{\text{atomic}} / T_{\text{multi_step}}) \times 100$$

Where:

T_{atomic} = financial operations executed atomically

$T_{\text{multi_step}}$ = total multi-step financial operations

A system with $AR < 80\%$ creates front-running surfaces (Vectors #2, #5).

8. Discussion

8.1 The Recursive Security Problem

The YerbaMate case study (Section 5.4.5) illustrates a deeper problem: the tools built to secure AI Agent $\times$ DeFi systems are themselves AI tools with DeFi interactions. An AI-powered auditor that ships with a reentrancy vulnerability is both a security tool and a security risk. This recursion—security infrastructure becoming part of the attack surface—is unique to the AI Agent $\times$ DeFi intersection and demands verification methodologies that apply to the security tools themselves.

8.2 The Time-of-Check to Time-of-Agent Problem

Traditional DeFi security addresses the time-of-check to time-of-use (TOCTOU) problem for on-chain state. AI agents introduce an additional temporal layer: time-of-check to time-of-agent-decision (TOCTAD). Between the moment the agent queries on-chain state and the moment the agent's LLM produces a decision and executes, the state may have changed. This window—measured in seconds for LLM inference versus milliseconds for on-chain execution—creates an asymmetry that favors attackers with programmatic execution speed.

8.3 The Centralization Paradox

The most effective defenses against several vectors require centralized components that may be philosophically incompatible with DeFi's decentralization ethos. Tool whitelists require a central authority to approve tools. Agent identity registries for Sybil resistance require a central issuer. Human-in-the-loop confirmation reintroduces centralized control. The AI Agent $\times$ DeFi security community must navigate this tension between effective security and decentralized architecture.

8.4 Limitations

This work has several limitations:

1. **Sample size:** Our empirical audit covers 5 protocols with 108 contracts—a fraction of the emerging AI Agent $\times$ DeFi ecosystem. As the ecosystem grows, additional vectors may emerge.
2. **Temporal scope:** We analyze protocols as of July 2026. Both AI agent capabilities and DeFi protocols evolve rapidly; vectors that are predictive today may become actively exploited within months.
3. **LLM specificity:** Our threat model assumes GPT-4-class LLM behavior. Different model architectures may introduce different interpretation gaps and exploitation patterns.

4. **Economic modeling:** We provide qualitative severity ratings but do not model the economic incentives that determine which vectors will be exploited first or most profitably. An economic analysis of attacker ROI across vectors is future work.
5. **Defense validation:** Our defense framework is analytically grounded but not empirically evaluated against real-world AI Agent × DeFi attacks (which have not yet occurred at scale). The framework should be treated as a starting point for defense research, not a validated security standard.

8.5 Future Work

Several research directions follow from this work:

1. **Automated agent fuzzing:** Develop a fuzzing framework that generates adversarial market conditions and tool responses to stress-test AI agent decision logic.
2. **Formal verification of agent-contract interactions:** Extend formal verification techniques (CertiK, Runtime Verification) to model the agent's decision process as part of the verification target, not just the contract's execution logic.
3. **Economic security analysis:** Model the attacker's ROI for each vector as a function of agent TVL, LLM latency, and MEV infrastructure availability, identifying the economic thresholds that determine which vectors will be exploited first.
4. **Cross-chain agent security:** Extend the taxonomy to cross-chain agent operations, where the agent manages positions across multiple blockchains through bridge protocols—an additional trust boundary.
5. **Agent security benchmarks:** Create a standardized benchmark suite (analogous to DeFiHackLabs' PoC contracts) for AI Agent × DeFi vulnerabilities, enabling reproducible evaluation of defense mechanisms.

8.6 Ecosystem Implications

The security of AI Agent × DeFi systems has implications beyond the protocols directly affected. If a widely deployed agent manager (e.g., an agent managing 10,000 users' yield positions) is exploited, the cascading effect—forced liquidations, oracle price crashes, panic withdrawals—could trigger systemic DeFi instability comparable to the 2022 Terra/LUNA collapse. The "too big to fail" problem, already present in DeFi through protocols like Lido and Aave, is amplified when a single agent controls capital across multiple protocols.

9. Conclusion

The marriage of AI agents and DeFi is inevitable—and so are the attacks that target the gap between them. This paper establishes the first systematic security study of the AI Agent × DeFi intersection, contributing an 8-vector attack taxonomy, empirical validation through audit of 108 smart contracts

across 5 protocols, an executable Foundry sandbox demonstrating 5 vectors, and a three-layer defense framework.

The central finding is that three CRITICAL vectors (#1 Tool Injection, #4 MCP MITM, #8 Autonomous Signing Theft) target infrastructure components that have no analog in traditional DeFi architecture. These vectors are currently unexploited not because the defenses are strong, but because the target infrastructure is too new. As MCP-based agent-DeFi integration expands, these vectors will become the primary attack surface unless the defense framework we propose is adopted preemptively.

The AI agent is neither a traditional user nor a traditional program—it occupies a unique position in the DeFi security landscape that combines human-like autonomy with programmatic determinism, and this dual identity demands a new subfield of security research that bridges smart contract auditing, AI safety, and protocol security. We hope this work catalyzes that subfield.

References

- [1] Chen, S. (2026a). A Comprehensive Taxonomy of DeFi Attack Patterns: 50 Vectors from 824 Incidents (2017–2026). Zenodo. DOI: 10.5281/zenodo.21405849.
- [2] Chen, S. (2026b). A Decade of DeFi Attacks: Pattern Evolution 2017–2026. Zenodo. DOI: 10.5281/zenodo.21403727.
- [3] Chen, S. (2026c). An Empirical Study of Model Context Protocol (MCP) Server Security: Taxonomy, Large-Scale Scanning, and Defense Framework. Zenodo. DOI: 10.5281/zenodo.21383532.
- [4] Chen, S. (2026d). When Type Hashes Lie: EIP-712 Implementation Errors in DeFi. Zenodo. DOI: 10.5281/zenodo.21405974.
- [5] Chen, S. (2026e). Prompt Injection is Not an AI Problem: Why MCP Tool Hardening Matters. Zenodo. DOI: 10.5281/zenodo.21388900.
- [6] Werner, S., Perez, D., Gudgeon, L., Klages-Mundt, A., Harz, D., & Knottenbelt, W. (2023). SoK: Decentralized Finance (DeFi) Attacks. IEEE S&P.
- [7] Atzei, N., Bartoletti, M., & Cimoli, T. (2017). A Survey of Attacks on Ethereum Smart Contracts. POST.
- [8] Zhou, L., Xiong, X., Ernstberger, J., Chaliasos, S., Wang, Z., Wang, Y., Qin, K., Wattenhofer, R., Song, D., & Gervais, A. (2023). SoK: Decentralized Finance (DeFi) Incidents—A Taxonomy, Categorization, and Common Vulnerabilities. ACM CCS.
- [9] Perez, F., & Ribeiro, I. (2022). Ignore Previous Prompt: Attack Techniques for Language Models. NeurIPS Workshop on ML Safety.
- [10] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. AISEC.
- [11] Ruan, Y., Dong, H., Wang, A., Pitis, S., Zhou, Y., Ba, J., Dubois, Y., Maddison, C. J., & Hashme, A. (2024). Identifying the Risks of LM Agents with an LM-Emulated Sandbox. ICLR.

- [12] Zou, A., Wang, Z., Kolter, J. Z., & Fredrikson, M. (2023). Universal and Transferable Adversarial Attacks on Aligned Language Models. arXiv:2307.15043.
- [13] Wei, A., Haghtalab, N., & Steinhardt, J. (2023). Jailbroken: How Does LLM Safety Training Fail? NeurIPS.
- [14] Daian, P., Goldfeder, S., Kell, T., Li, Y., Zhao, X., Bentov, I., Breidenbach, L., & Juels, A. (2020). Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability. IEEE S&P.
- [15] Qin, K., Zhou, L., & Gervais, A. (2021). Quantifying Blockchain Extractable Value: How Dark is the Forest? IEEE S&P.
- [16] Anthropic. (2024). Model Context Protocol Specification. <https://modelcontextprotocol.io>.
- [17] Clicks Protocol. (2026). Agent commerce settlement router. <https://github.com/clicks-protocol>.
- [18] Cairn Protocol. (2026). Standardized checkpoint & recovery for AI agents. <https://github.com/swarmproof/cairn-protocol>.
- [19] PropFund. (2026). Decentralized prop trading for AI agents. <https://github.com/NO7r34L/PropFund.eth>.
- [20] Willison, S. (2023). Delimiters Won't Save You From Prompt Injection. <https://simonwillison.net>.

All findings, sandbox code, audit reports, and supplementary materials available at: <https://github.com/shunfeng8421/defi-hack-memo>

Foundry sandbox code available at: </paper/09-ai-agent-defi/sandbox/>

Responsible disclosure timeline: Findings reported to protocol teams July 2026; 90-day disclosure window.